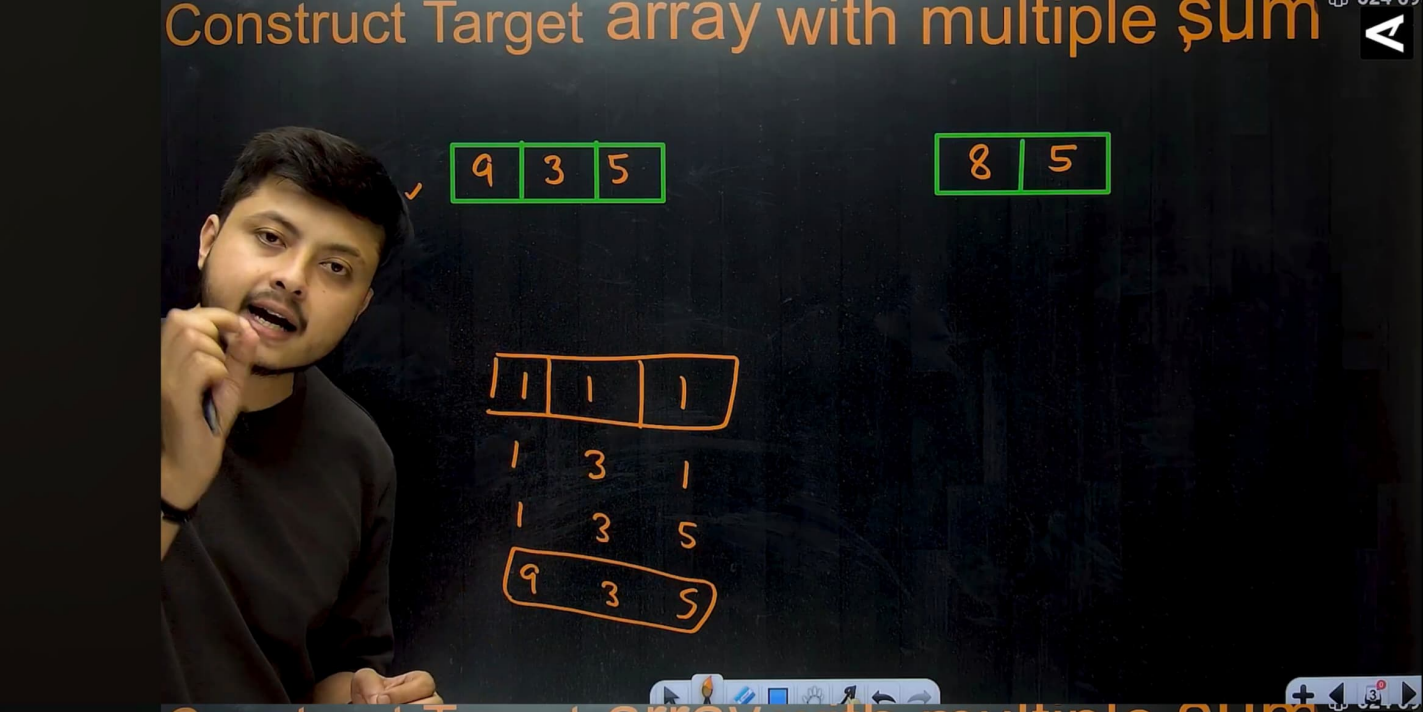
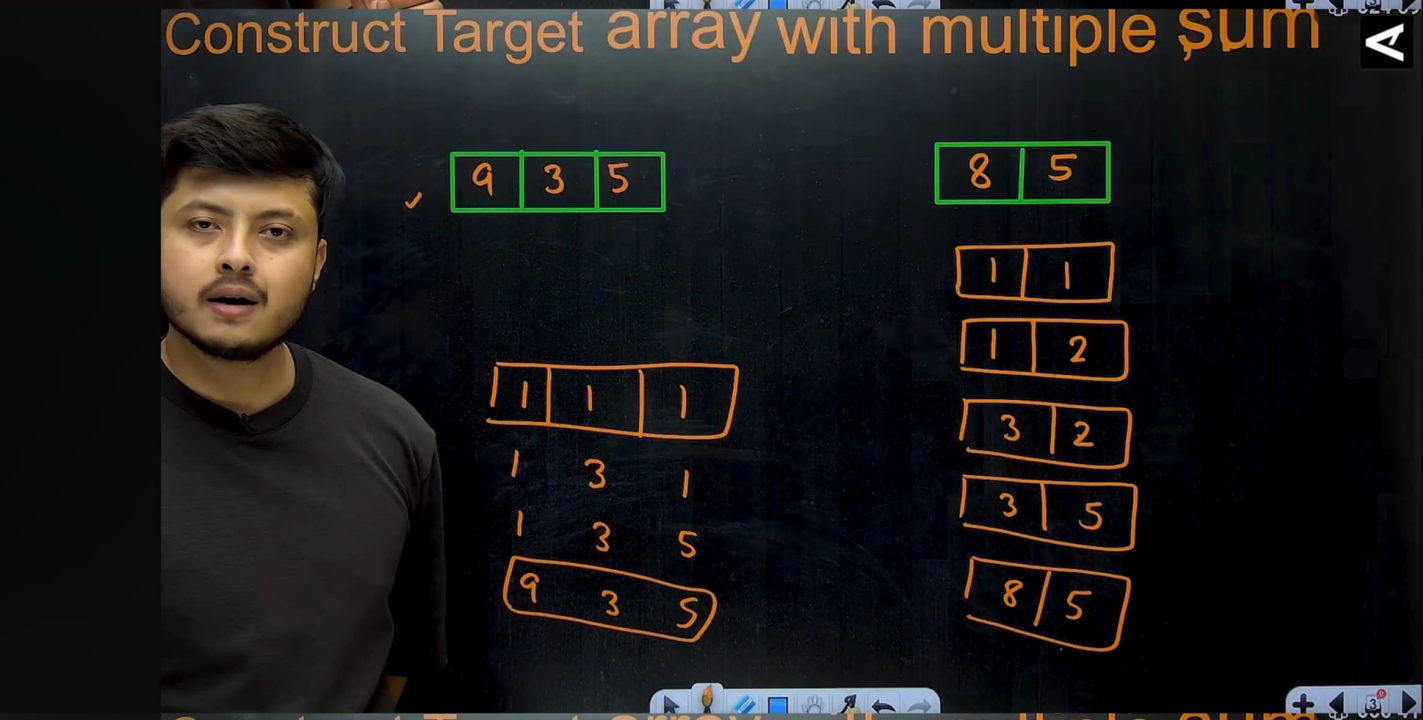
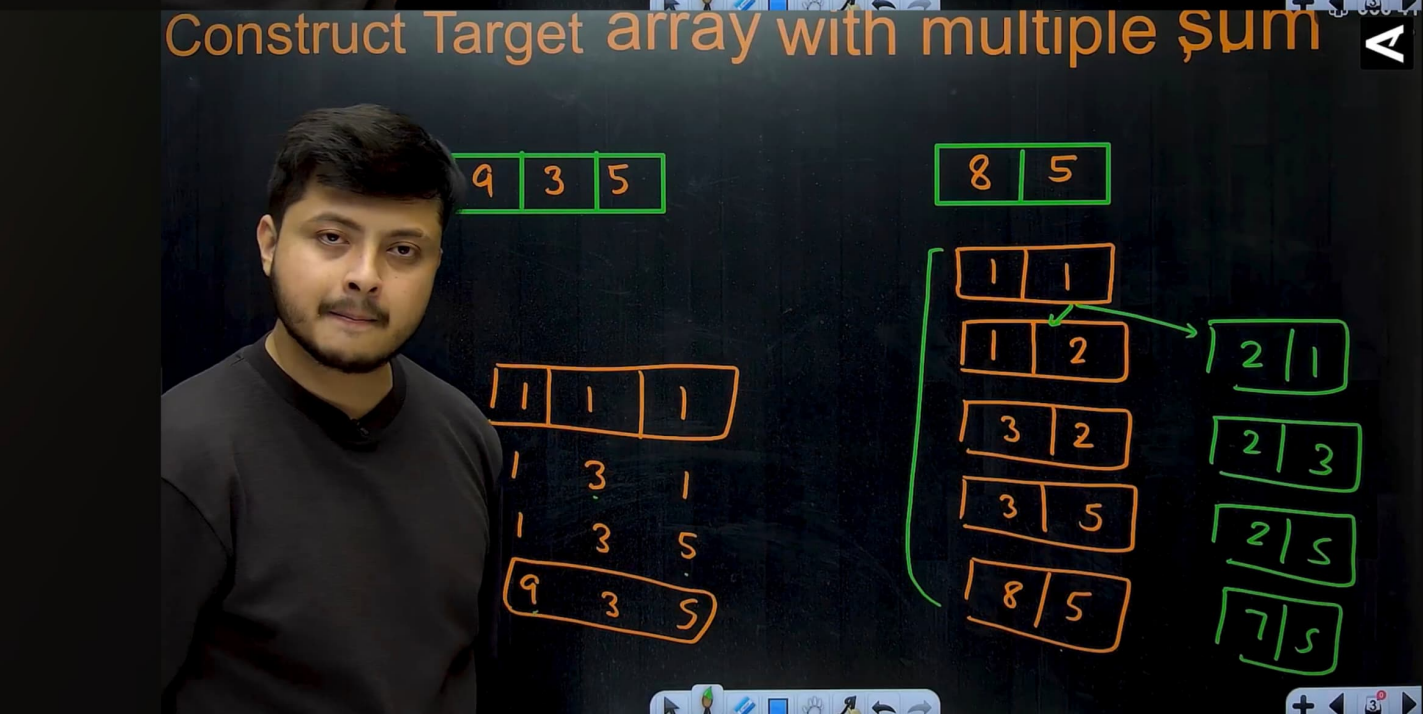
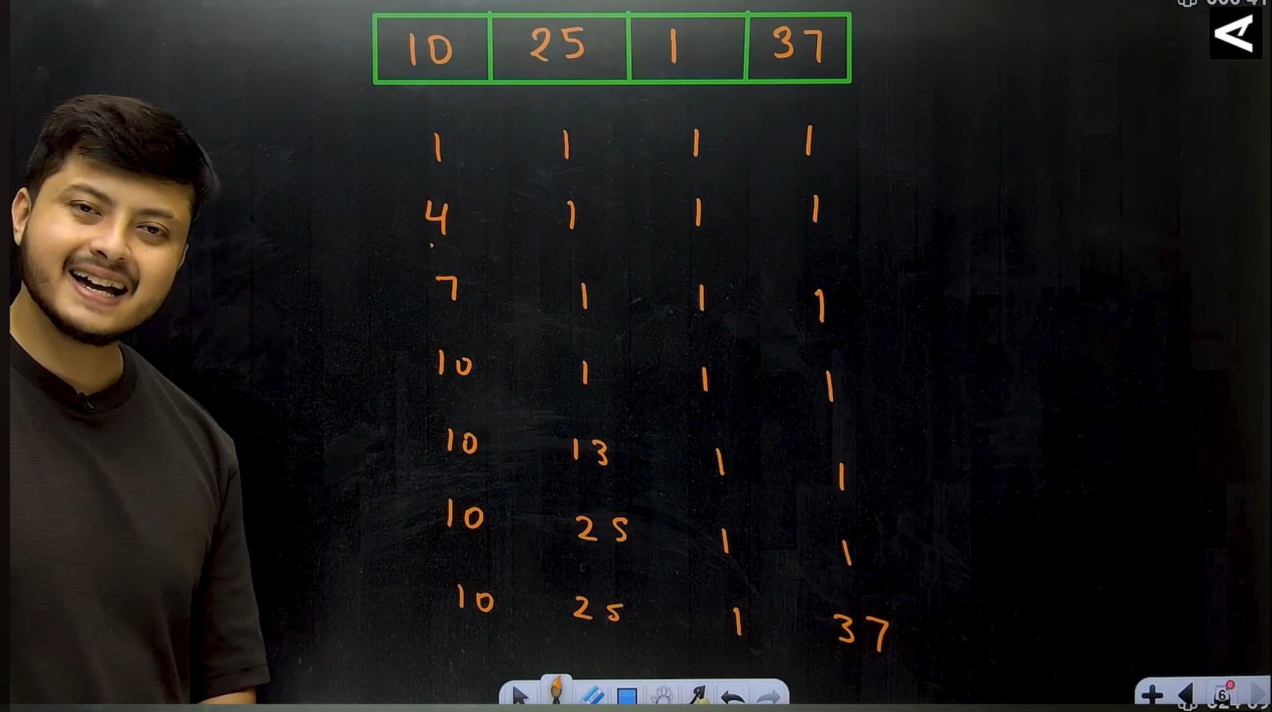
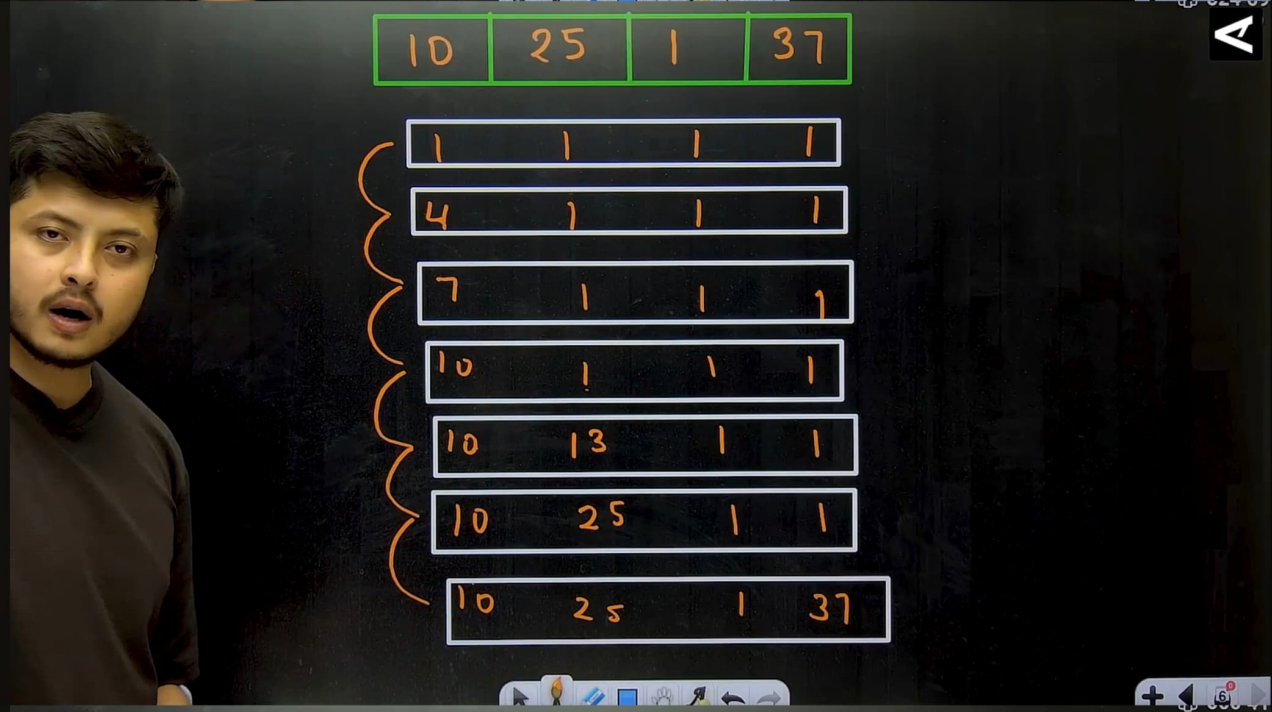
Construct Target array with multiple sum

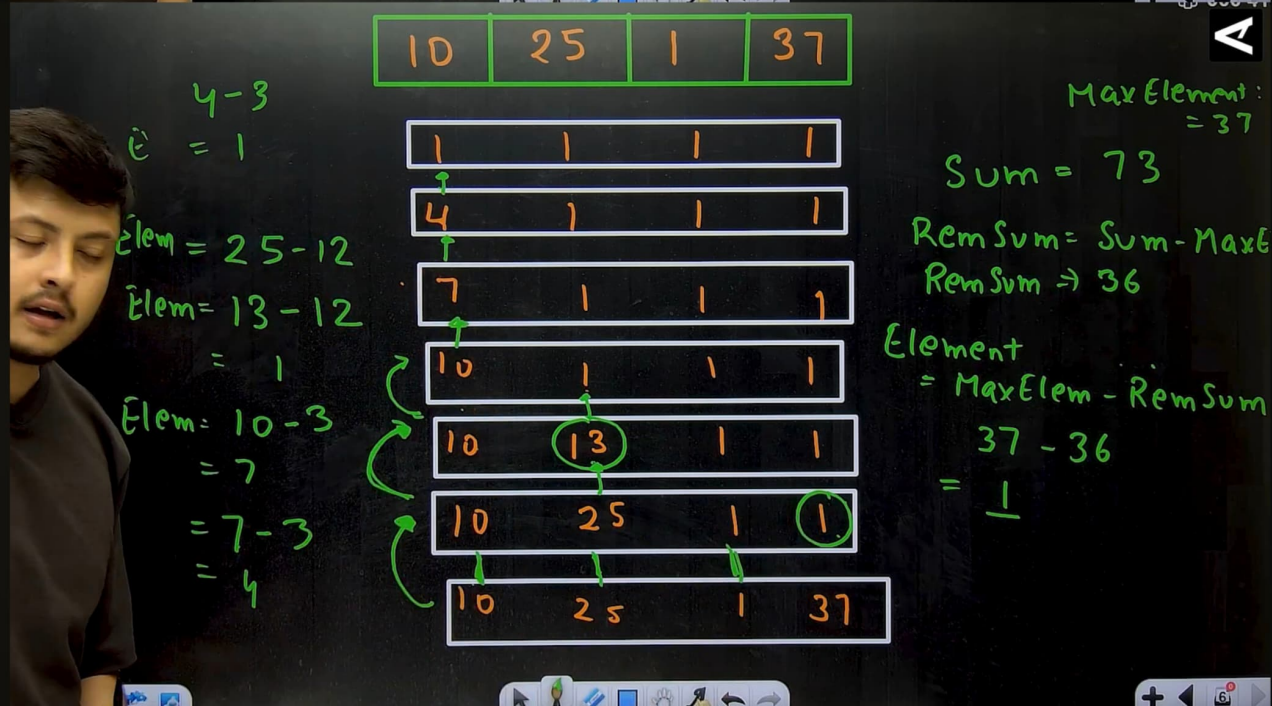

Construct Target array with multiple sum
9 3 5
8 5
1 1 1
1 3 1
1 3 5
9 3 5

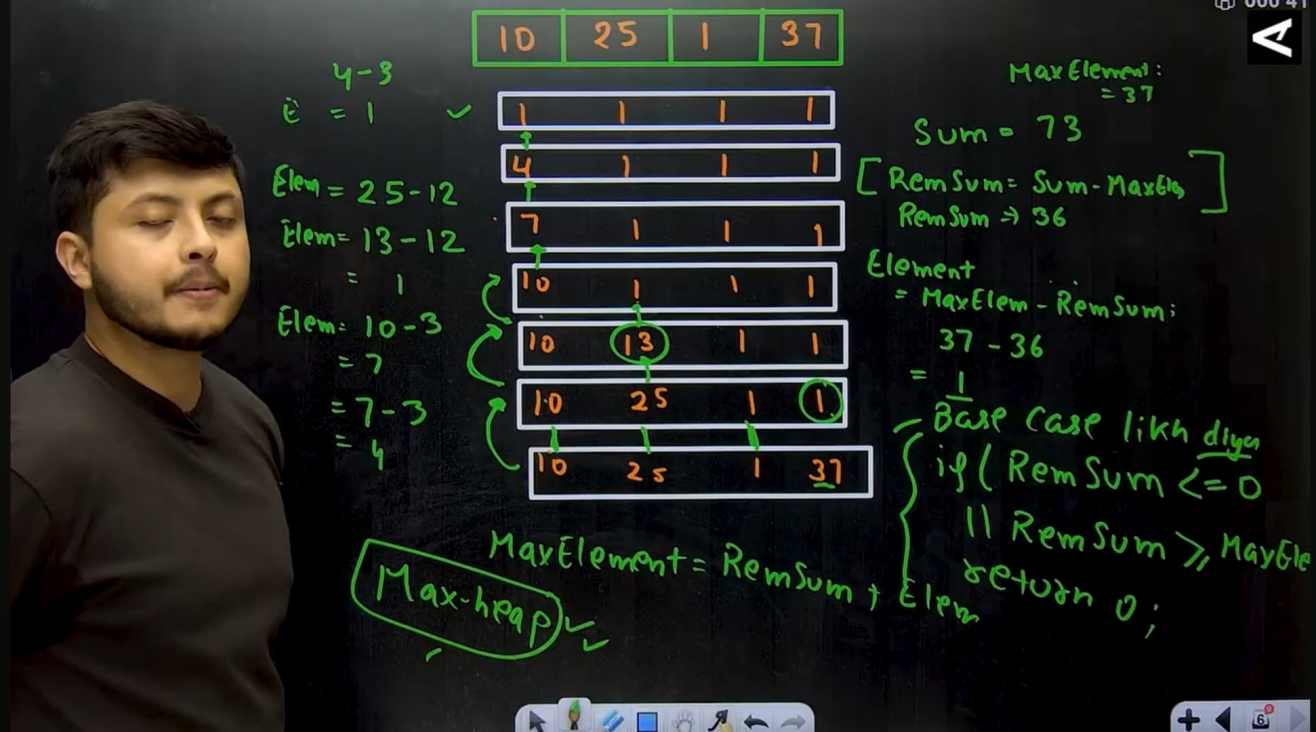

Construct Target array with multiple sum
9 3 5
8 5
1 1 1
1 3 1
1 3 5
9 3 5
1 1
1 2
3 2
3 5
8 5


Construct Target array with multiple sum
9 3 5
8 5
1 1 1
1 3 1
1 3 5
9 3 5
1 1
1 2
3 2
3 5
8 5
2 1
2 3
2 5
7 5


10 25 1 37
1 1 1 1
4 1 1 1
7 1 1 1
10 1 1 1
10 13 1 1
10 25 1 1
10 25 1 37


10 25 1 37
1 1 1 1
4 1 1 1
7 1 1 1
10 1 1 1
10 13 1 1
10 25 1 1
10 25 1 37


10 25 1 37
4-3
= 1
Elem = 25-12
Elem = 13-12
= 1
Elem = 10-3
= 7
= 7-3
= 4
MaxElement:
= 37
Sum = 73
RemSum = Sum - MaxE
RemSum → 36
Element
= MaxElem - RemSum
37 - 36
= 1
10 25 1 37
1 1 1 1
4 1 1 1
7 1 1 1
10 1 1 1
10 13 1 1
10 25 1 1
10 25 1 37


10 25 1 37
4-3
= 1
Elem = 25-12
Elem = 13-12
= 1
Elem = 10-3
= 7
= 7-3
= 4
1 1 1 1
4 1 1 1
7 1 1 1
10 1 1 1
10 13 1 1
10 25 1 1
10 25 1 37
MaxElement = RemSum + Elem
Max-heap
MaxElement : = 37
Sum = 73
[RemSum = Sum - MaxEle
RemSum → 36]
Element = MaxElem - RemSum;
37 - 36
= 1
Base case likh diya
if (RemSum <= 0
|| RemSum >= MaxEle
return 0;

Problem List

Run Submit

Description

1354. Construct Target Array With Multiple Sums

Solved

Hard Topics Companies Hint

You are given an array `target` of n integers. From a starting array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
```

2K 7

Editorial Solutions Submissions

Code

C++ Auto

```
class Solution {
public:
    bool isPossible(vector<int>& target) {

    }
};
```

Saved Ln 4

Testcase Test Result

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
Explanation: Start with arr = [1, 1, 1]
[1, 1, 1], sum = 3 choose index 1
[1, 3, 1], sum = 5 choose index 2
[1, 3, 5], sum = 9 choose index 0
[9, 3, 5] Done
```

Example 2:

2K 7

Editorial Solutions Submissions

Code

C++ Auto

```
class Solution {
public:
    bool isPossible(vector<int>& target) {
        // maxheap
        priority_queue<int>p;
        int sum = 0;

        for(int i=0;i<target.size();i++)
        {
            p.push(target[i]);
            sum+=target[i];
        }
    }
};
```

Saved Ln 11,

Testcase Test Result

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
Explanation: Start with arr = [1, 1, 1]
[1, 1, 1], sum = 3 choose index 1
[1, 3, 1], sum = 5 choose index 2
[1, 3, 5], sum = 9 choose index 0
[9, 3, 5] Done
```

Example 2:

2K 7

Editorial Solutions Submissions

Code

C++ Auto

```
        for(int i=0;i<target.size();i++)
        {
            p.push(target[i]);
            sum+=target[i];
        }

        int MaxEle , RemSum , Element;

        while(p.top()!=1) //
        {


        }


        return 1;
```

Saved Ln 16,

Testcase Test Result

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
Explanation: Start with arr = [1, 1, 1]
[1, 1, 1], sum = 3 choose index 1
[1, 3, 1], sum = 5 choose index 2
[1, 3, 5], sum = 9 choose index 0
[9, 3, 5] Done
```

Example 2:

```
Input: target = [1,1,1,2]
```

2K 7

Editorial Solutions Submissions

Code

C++ Auto

```
12      }
13
14      int MaxEle , RemSum , Element;
15
16      while(p.top()!=1) //
17      {
18          MaxEle = p.top();
19          p.pop();
20          RemSum = sum - MaxElem;
21
22          // MaxElem = RemSum + Element
23          // Edges case
24          if(RemSum<=0 || RemSum>=MaxElem)
25          return 0;
26
27      }
```

Saved Ln 25,

Testcase Test Result

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
Explanation: Start with arr = [1, 1, 1]
[1, 1, 1], sum = 3 choose index 1
[1, 3, 1], sum = 5 choose index 2
[1, 3, 5], sum = 9 choose index 0
[9, 3, 5] Done
```

Example 2:

```
Input: target = [1,1,1,2]
```

2K 7

Editorial Solutions Submissions

Code

C++ Auto

```
16      while(p.top()!=1) //
17      {
18          MaxEle = p.top();
19          p.pop();
20          RemSum = sum - MaxElem;
21
22          // MaxElem = RemSum + Element
23          // Edges case
24          if(RemSum<=0 || RemSum>=MaxElem)
25          return 0;
26
27          Element = MaxEle - RemSum;
28          sum = RemSum + Element;
29          p.push(Element);
30
31      }
```

Saved Ln 30,

Testcase Test Result

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
Explanation: Start with arr = [1, 1, 1]
[1, 1, 1], sum = 3 choose index 1
[1, 3, 1], sum = 5 choose index 2
[1, 3, 5], sum = 9 choose index 0
[9, 3, 5] Done
```

Example 2:

```
Input: target = [1,1,1,2]
```

2K 7

Editorial Solutions Submissions

Code

C++ Auto

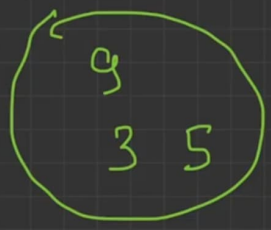
```
19          p.pop();
20          RemSum = sum - MaxElem;
21
22          // MaxElem = RemSum + Element
23          // Edges case
24          if(RemSum<=0 || RemSum>=MaxElem)
25          return 0;
26
27          Element = MaxEle - RemSum;
28          sum = RemSum + Element;
29          p.push(Element);
30      }
31
32      return 1;
33
34  }
```

Saving... Ln 29,

Testcase Test Result

[9, 3, 5]

Sum = 17



MaxElem = 9

RemSum = 8

[9, 3, 5]

[1, 3, 5]

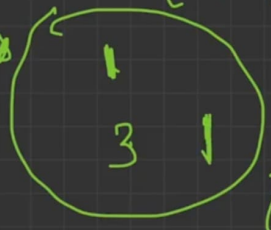
[1, 3, 1]

Max = 5

Sum = 9

Rem = 4

Elem = 1



[1, 1, 1]

Sum = RemSum + Ele

Sum = 8 + 1

Sum = 17 9

MaxElem = 9

RemSum = 8

Element = 9 - 8

= 1.

[9, 3, 5]

[1, 3, 5]

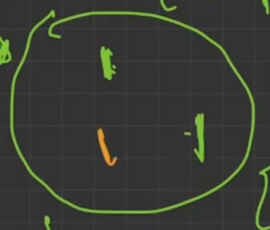
[1, 3, 1]

Max = 5

Sum = 9

Rem = 4

Elem = 1



[1, 1, 1]

Sum = 4 + 1

Max = 3

Sum = 5

Rem = 2

Elem = 1

Sum = RemSum + Ele

Sum = 8 + 1

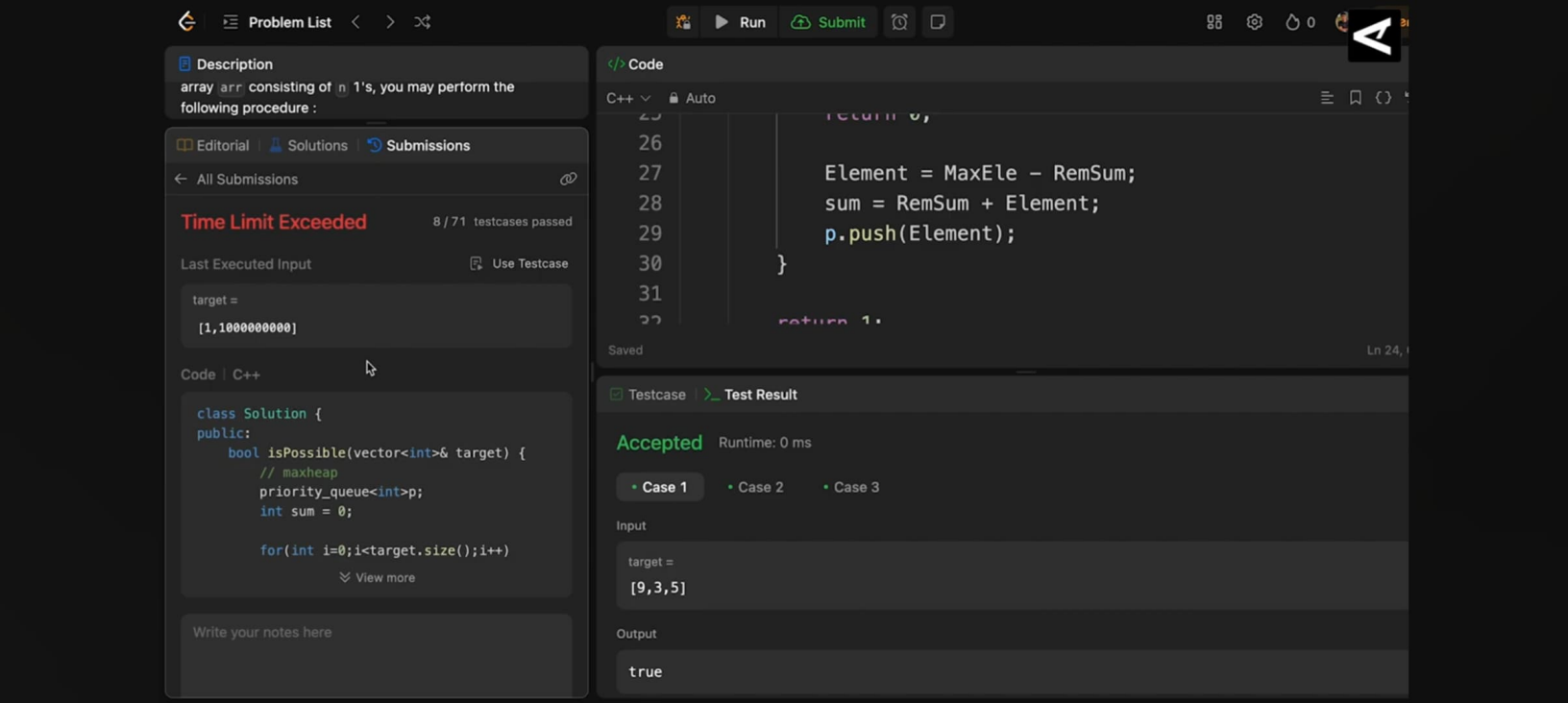
Sum = 17 9

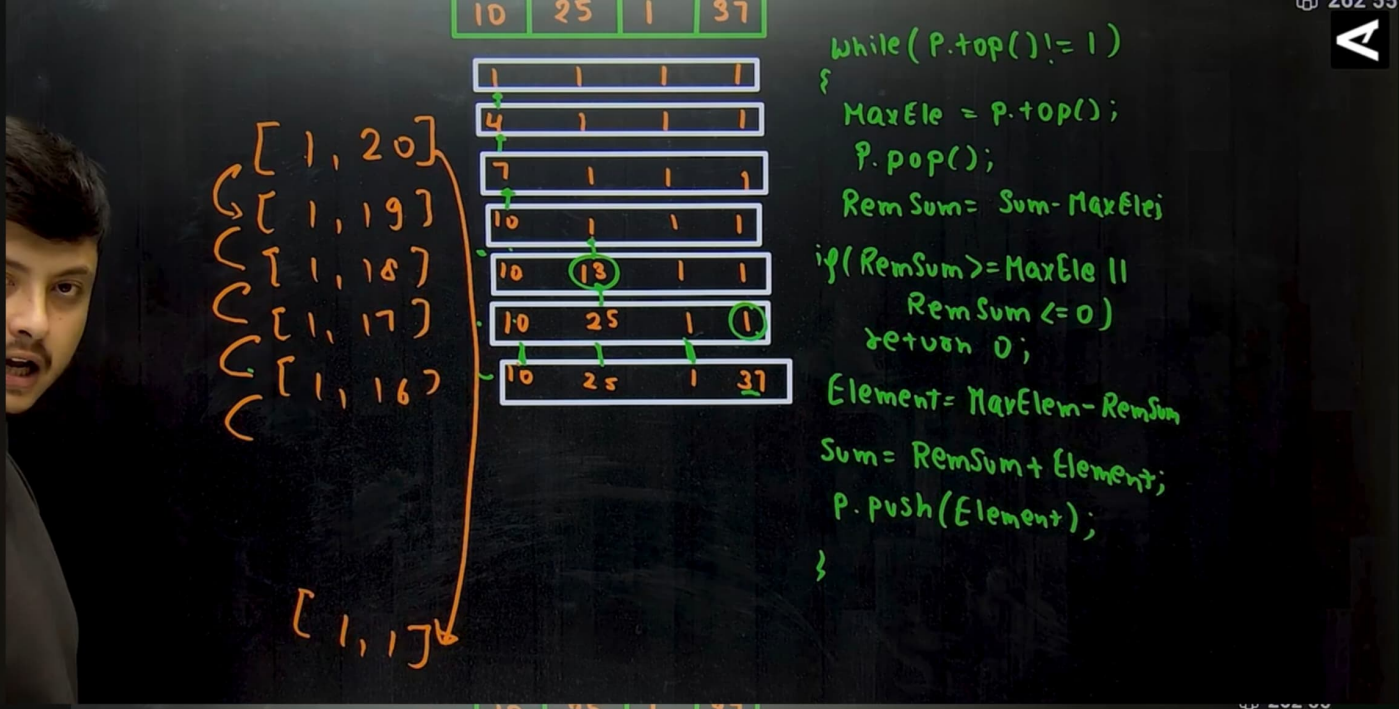
MaxElem = 9

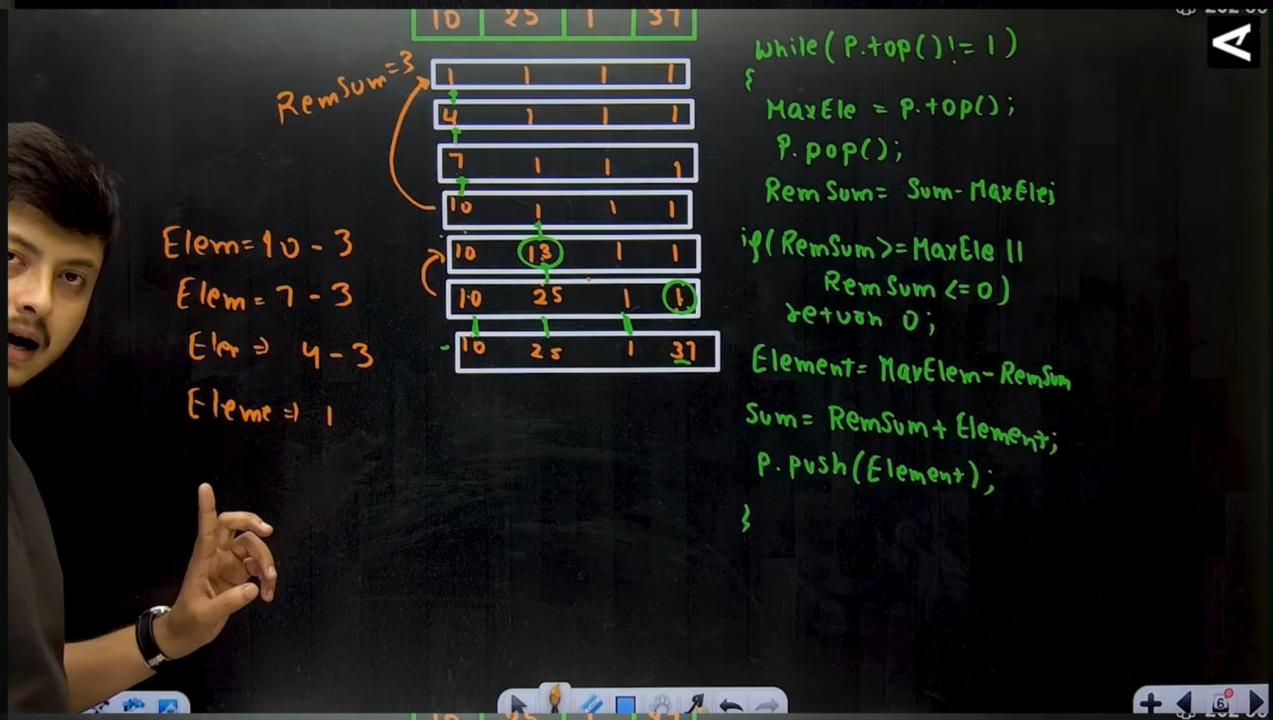
RemSum = 8

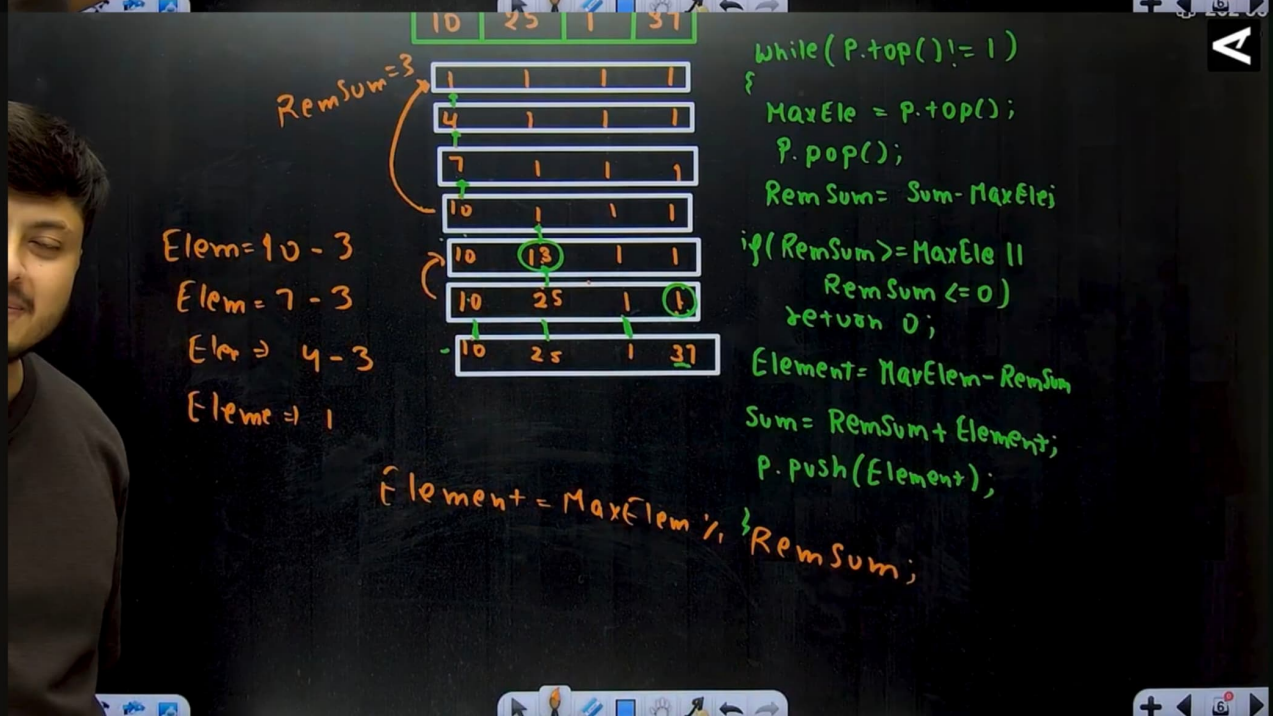
Element = 9 - 8

= 1.


Problem List
Run
Submit
Description
array arr consisting of n 1's, you may perform the following procedure :
Editorial
Solutions
Submissions
All Submissions
Time Limit Exceeded
8 / 71 testcases passed
Last Executed Input
Use Testcase
target =
[1,1000000000]
Code | C++
class Solution {
public:
bool isPossible(vector<int>& target) {
// maxheap
priority_queue<int>p;
int sum = 0;
for(int i=0;i<target.size();i++)
View more
Write your notes here
Code
C++
Auto
Element = MaxEle - RemSum;
sum = RemSum + Element;
p.push(Element);
}
Saved
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
target =
[9,3,5]
Output
true


10 25 1 37
[1, 20]
[1, 19]
[1, 18]
[1, 17]
[1, 16]
[1, 1]
While(P.top()!=1)
{
MaxEle = P.top();
P.pop();
RemSum= Sum-MaxElej
if(RemSum>=MaxEle ||
RemSum<=0)
return 0;
Element= MaxElem-RemSum
Sum= RemSum+Element;
P.push(Element);
}


RemSum=3
Elem=10-3
Elem=7-3
Elem=4-3
Eleme=1
While(P.top()!=1)
{
MaxEle = P.top();
P.pop();
RemSum= Sum-MaxElej
if(RemSum>=MaxEle ||
RemSum<=0)
return 0;
Element= MaxElem-RemSum
Sum= RemSum+Element;
P.push(Element);
}

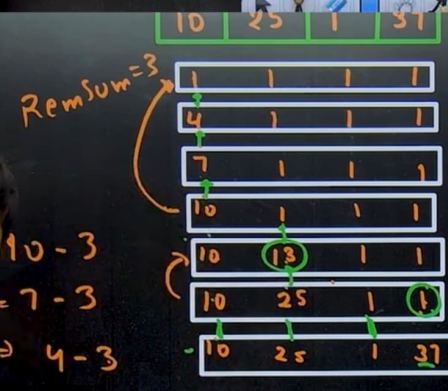

RemSum=3
Elem=10-3
Elem=7-3
Elem=4-3
Eleme=1
While(P.top()!=1)
{
MaxEle = P.top();
P.pop();
RemSum= Sum-MaxElej
if(RemSum>=MaxEle ||
RemSum<=0)
return 0;
Element= MaxElem-RemSum
Sum= RemSum+Element;
P.push(Element);
}
Element = MaxElem%; }RemSum;

20 - 5
15 - 5
10 - 5
5 - 5
0 =

20 % 5 = 0

21 - 5
16 - 5
11 - 5
6 - 5
1

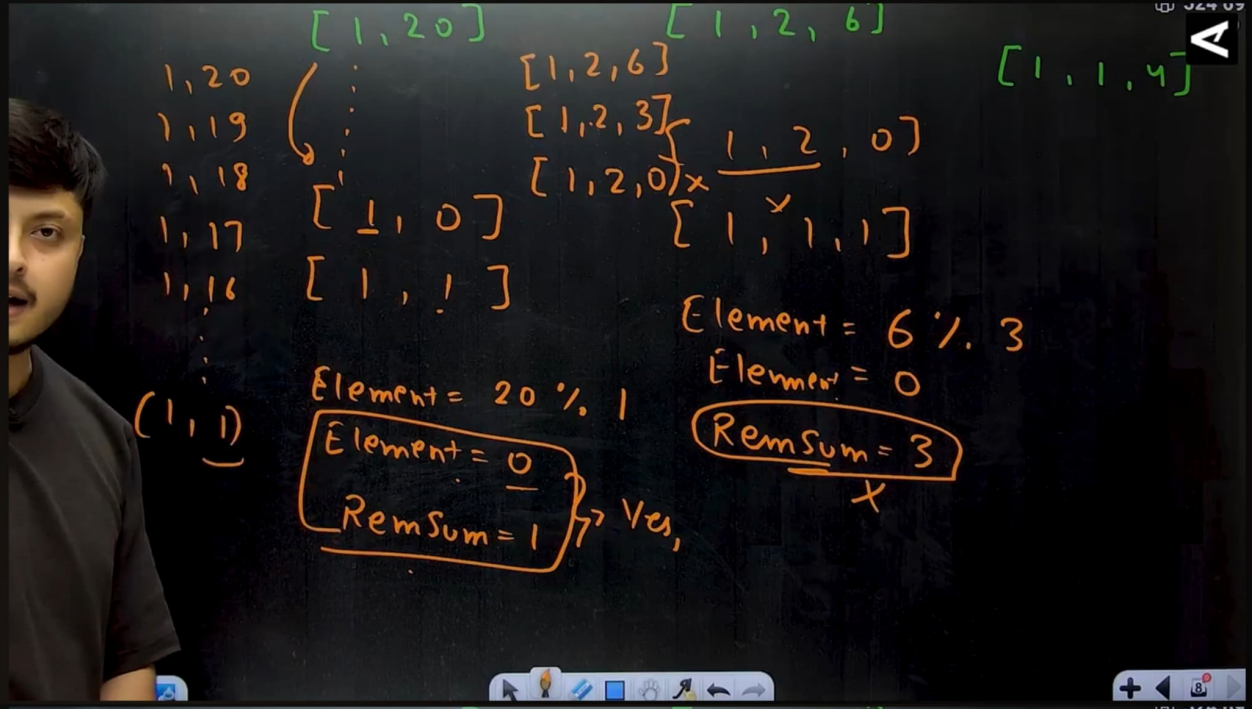
21 % 5 = 1

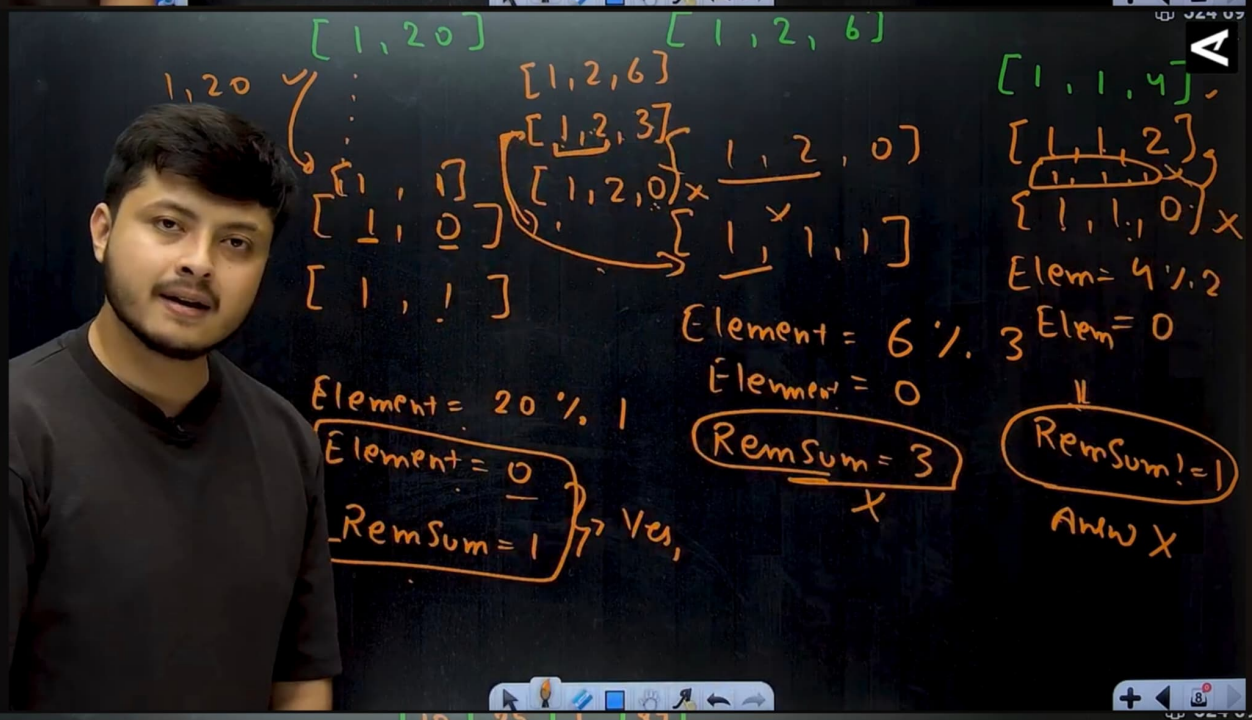


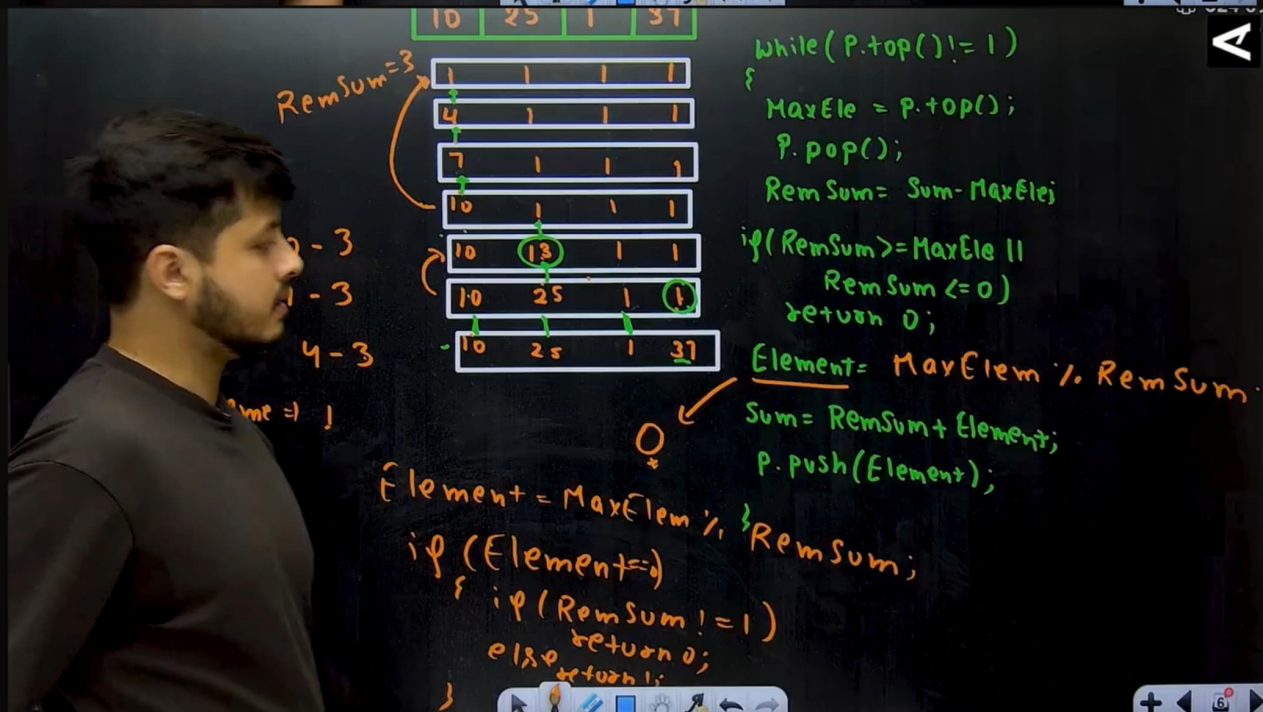

```
while(P.top()!=1)
{
  MaxEle = P.top();
  P.pop();
  RemSum= Sum-MaxEle;
if(RemSum>=MaxEle ||
     RemSum<=0)
   return 0;
Element= MaxElem % RemSum;
Sum= RemSum+Element;
P.push(Element);
}
```

Element = MaxElem % RemSum;

0 - RemSum-1


[1,20]
[1,2,6]
[1,1,4]
1,20
1,19
1,18
1,17
1,16
[1,2,6]
[1,2,3]
[1,2,0]
[1,2,0]
[1,1,1]
[1,0]
[1,1]
(1,1)
Element = 20 % 1
Element = 0
RemSum = 1
Yes,
Element = 6 % 3
Element = 0
RemSum = 3


[1,20]
[1,2,6]
[1,1,4]
1,20
[1,2,6]
[1,2,3]
[1,2,0]
[1,2,0]
[1,1,1]
[1,1]
[1,0]
[1,1]
Element = 20 % 1
Element = 0
RemSum = 1
Yes,
Element = 6 % 3
Element = 0
RemSum = 3
Elem = 4 % 2
Elem = 0
RemSum != 1
Ans = X


10 25 1 31
RemSum=3
10 13 1 1
10 25 1 1
10 25 1 31
while(P.top()!=1)
{
MaxEle = P.top();
P.pop();
RemSum = Sum-MaxEle;
if(RemSum>=MaxEle ||
RemSum<=0)
return 0;
Element = MaxElem % RemSum;
Sum = RemSum+Element;
P.push(Element);
}
Element = MaxElem % RemSum;
if(Element==0)
if(RemSum!=1)
return 0;
else return 1;

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

```
Input: target = [9,3,5]
Output: true
Explanation: Start with arr = [1, 1, 1]
[1, 1, 1], sum = 3 choose index 1
[1, 3, 1], sum = 5 choose index 2
[1, 3, 5], sum = 9 choose index 0
[9, 3, 5] Done
```

Example 2:

```
Input: target = [1,1,1,2]
```

2K 7

Editorial | Solutions | Submissions

Code

C++ Auto

```
26
27          // changes will occur here
28          Element = MaxEle % RemSum;
29          if(Element==0)
30          {
31              if(RemSum!=1)
32              return 0;
33              else
34              return 1;
35          }
36          sum = RemSum + Element;
```

Saved Ln 32,

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

Problem List

Run Submit

Description

array `arr` consisting of `n` 1's, you may perform the following procedure :

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

2K 7

Editorial | Solutions | Submissions

All Submissions

Runtime Error 54 / 71 testcases passed

```
Line 11: Char 16: runtime error: signed integer ov
erflow: 2136851853 + 618310025 cannot be represent
ed in type 'int' (solution.cpp)
SUMMARY: UndefinedBehaviorSanitizer: undefined-beh
avior prog_joined.cpp:20:16
```

Last Executed Input Use Testcase

target =

Code

C++ Auto

```
1  class Solution {
2  public:
3      bool isPossible(vector<int>& target) {
4          // maxheap
5          priority_queue<long long>p;
6          int sum = 0;
7
8          for(int i=0;i<target.size();i++)
9          {
10             p.push(target[i]);
11             sum+=target[i];
```

Saving... Ln 5,

Testcase | Test Result

Accepted Runtime: 3 ms

Case 1 Case 2 Case 3

Input

Take long long instead of int.

- let `x` be the sum of all elements currently in your array.
- choose index `i`, such that `0 <= i < n` and set the value of `arr` at index `i` to `x`.
- You may repeat this procedure as many times as needed.

Return `true` *if it is possible to construct the* `target` *array from* `arr`*, otherwise, return* `false`.

Example 1:

2K 7

Editorial | Solutions | Submissions

All Submissions

Accepted

submitted at May 05, 2

Editorial Solution

Runtime

34 ms

Beats 37.65% of users with C++

Memory

34.86 MB

Beats 18.82% of users with C++

```
25          return 0;
26
27          // changes will occur here
28          Element = MaxEle % RemSum;
29          if(Element==0)
30          {
31              if(RemSum!=1)
32              return 0;
33              else
34              return 1;
35          }
36          sum = RemSum + Element;
37          p.push(Element);
38      }
```

Saved Ln 35,

Testcase | Test Result

Accepted Runtime: 3 ms

Case 1 Case 2 Case 3

Input

target =

[9,3,5]